

Express JS

Miscellaneous Tasks

[For logical Refence]

Multer

Example - 1

Write an express js script to configure the **multer** middleware. Perform following tasks.

- 1) Create a **html** file named "file.html". This file contains heading(h3) "Upload your CV" in red color. And, a form with input type file(to browse and select file) and submit(to upload the file).
- 2) Create a js file named "file.js" and link this js and html file to browse html file on "/home" page.
- 3) After uploading a file display message on "/upload" page "(file original name) has been uploaded".
- 4) Save uploaded files to specific directory named "upload". And in this folder file must be stored in format of "lju-file.pdf" where "lju" is the field name.

file.html [both files in same folder]

```
<body>
  <h3 style="color:red">Upload your CV</h3>
  <form action="/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="lju">
    <input type="submit" value="Submit">
  </form>
</body>
```

file.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
const app = express();
app.use(express.static(__dirname, {index: "file.html"}));

// SET STORAGE
var storage = multer.diskStorage({
  destination: "upload",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-file.pdf")
  }
})
```

```
})  
var upload = multer({ storage: storage })  
app.post('/upload', upload.single('lju'), (req, res) => {  
  const file = req.file  
  console.log(file)  
  if (file) {  
    res.send("<h1>File <span style='color:red>'"+ file.originalname + "'</span> has been  
uploaded in <span style='color:red>' + file.destination + " </span>folder")  
  }  
})  
app.listen(6788);
```

In browser

Upload your CV

Chapter 1 .pdf

File 'Chapter 1 .pdf' has been uploaded in upload folder

In console

Output of console.log(file)

```
{  
  fieldname: 'lju',  
  originalname: 'Chapter 1 .pdf',  
  encoding: '7bit',  
  mimetype: 'application/pdf',  
  destination: 'upload',  
  filename: 'lju-file.pdf',  
  path: 'upload\\lju-file.pdf',  
  size: 1200577  
}
```

Example -2

Write an express js script to configure the multer middleware. Perform following tasks.

- 1) Create a html form file named "form.html" in public folder. This file contain centrally oriented heading(h3) "Upload your File" in red color with background-color aqua. Along with choose file option(to browse and select file) and submit button(to upload the file). (Must use external css having name "effect.css" in public folder)
- 2) Create a js file to show result after uploading any type of file, message should be displayed on "/upload" page "(file original name) has been uploaded".(Css effect must include while running js code)
- 3) Save uploaded files to specific directory named "File". And in this folder file must be stored in format of "data-file.pdf" where "data" is the field name.

App.js (In src folder)

```
const express = require('express');
const multer = require('multer');
const path = require('path');

// Initialize the app
const app = express();

// Serve static files from the public folder
app.use(express.static("../public", {index: "form.html"}))

// Set up storage using multer
const storage = multer.diskStorage({
  destination: "File",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-file.pdf");
  }
});

// Initialize upload middleware
const upload = multer({ storage: storage }).single('data')

// Handle file upload
app.post('/upload', upload, (req, res) => {
  if (!req.file) {
    return res.status(400).send('No file uploaded.');
```

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

form.html (In public folder)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>File Upload</title>
  <link rel="stylesheet" href="/effect.css">
</head>
<body>
  <div class="container">
    <h3>Upload your File</h3>
    <form action="/upload" method="post" enctype="multipart/form-data">
      <input type="file" name="data" required>
      <button type="submit">Submit</button>
    </form>
  </div>
</body>
</html>
```

effect.css(In public folder)

```
body { background-color: aqua;}

h3 { color: red;}
//optional
.container { text-align: center;}
```

Nodemailer

Example-1:

Write a code to set up nodemailer in Express.js. Sender email id: "lju@gmail.com". Receiver email ids: "student@gmail.com and faculty@gmail.com". Mail subject should be "LJ University" Mail body contains "Welcome Student" in h3 tag and in table display data Date 28/06/23, Exam name FSD-2.

Solution:

```
const nodemailer = require('nodemailer');
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'lju@gmail.com',
    pass: 'yourpassword'
  }
});

const mailOptions = {
  from: 'lju@gmail.com',
  to: 'student@gmail.com, faculty@gmail.com',
  subject: 'LJ University',
  html: '<h3>welcome student</h3><table border="1px"><tr><th>date</th><th>Exam<br>name</th></tr><tr><td> 28/06/23</td><td>FSD-2</td></tr>'
};
transporter.sendMail(mailOptions, (error, info) => {
  if (error) {
    console.log(err)
  }
  console.log("mail send")
});
```

Example-2:

Perform the following tasks as asked: 1) Create a HTML file for response form and this file should be loaded on home('/') page. •Fields are : Name, Email and Submit button. 2) Once Response is submitted, message "Thank you for your response." Will be displayed on page '/response' and also send mail to the entered email id with the submitted response.

```
App.js
const express = require('express');
const nodemailer = require('nodemailer');

const app = express();
```

```
app.use(express.urlencoded({ extended: true }));

// serve HTML file
app.use(express.static(__dirname));

// configure mail transporter (Gmail example)
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'your_email@gmail.com',    // your email
    pass: 'your_app_password'       // app password (not normal password)
  }
});

// handle form submission
app.post('/response', (req, res) => {
  const { name, email } = req.body;

  // email content
  const mailOptions = {
    from: 'your_email@gmail.com',
    to: email,
    subject: 'Response Received',
    text: `Hello ${name},\n\nThank you for your response.`
  };

  // send email
  transporter.sendMail(mailOptions, (error, info) => {
    if (error) {
      console.log(error);
    } else {
      console.log("Email sent: " + info.response);
    }
  });

  // response page
  res.send('<h2>Thank you for your response.</h2>');
});

// start server
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Response Form</title>
```

```

</head>
<body>

<h2>Response Form</h2>

<form action="/response" method="POST">
  Name: <input type="text" name="name" required><br><br>
  Email: <input type="email" name="email" required><br><br>
  <input type="submit" value="Submit">
</form>

</body>
</html>

```

Example -3

Perform the following tasks as asked.

- Create a HTML file for feedback form and this file should be loaded on home('/') page.
- Fields are :
name, email, dropdown for the feedback ,text area for the additional comments, and submit button.
- Once feedback submitted, message “Thank you for your feedback.” Will be displayed on page '/feedback' and also **send mail** to the entered email id with the submitted feedback data. (Data can be submitted using get/post method)

Src/mail_form.js

```

const expr = require("express");
const app = expr();
const nm = require("nodemailer");

// static folder
app.use(expr.static("../public", { index: "mail.html" }));

app.get("/feedback", (req, res) => {
  // store query values
  const { mail, uname, feedback, comments } = req.query;

  // transporter
  const trans = nm.createTransport({
    host: "smtp.gmail.com",
    port: 465,
    secure: true,
    auth: {
      user: "sender@gmail.com",
      pass: "2stepverificationkey"
    }
  });
});

```

```

    }
  });

  // mail options using template literal
  const mailoption = {
    from: "noreply@gmail.com <noreply>",
    to: mail,
    bcc: "bcc@gmail.com",
    subject: "Feedback response",
    html: `<h1>Hello ${uname}!</h1>
      <h2> Thank you for submitting the feedback.<br>
        Your feedback:
        <span style="color:red">${feedback}</span>
      </h2>
      <h3>Additional comments: ${comments}</h3> `
  };

  // send mail
  trans.sendMail(mailoption, (err, info) => {
    if (err) return console.error(err);
    console.log(info);
  });
  res.send("Thank you for your feedback.");
});
app.listen(5001);

```

/public/mail.html

```

<html>
  <head><title>Feedback Form</title></head>
  <body>
    <form action="/feedback" method="get">
      Name: <input type="text" name="uname"/>
      Email: <input type="text" name="mail"/>
      Feedback: <select name="feedback">
        <option value="bad">Bad</option>
        <option value="good">Good</option>
        <option value="verygood">Very Good</option>
        <option value="excellent">Excellent</option>
      </select>
      Comments: <textarea name="comments" rows="10" cols="15"
placeholder="Comments"></textarea>
      <input type="submit" value="Submit"/>
    </form>
  </body>
</html>

```


Ejs

Example -1

Write an Express.js application to perform the following tasks:

1) Create an EJS file named contact.ejs that displays a heading "Contact Form" and contains a form with:

- Text input for Name
- Email input for Email
- Submit button

2) Render this EJS file on the /contact route.

3) Handle form submission using the POST method on the /submit route.

4) After form submission, display a message on the browser in the format:

"Thank you (Name), we have received your email (Email)." Through ejs file.

views/contact.ejs

```
<h2>Contact Form</h2>
<form action="/submit" method="POST">
  Name: <input type="text" name="name" required><br><br>
  Email: <input type="email" name="email" required><br><br>
  <input type="submit" value="Submit">
</form>
```

app.js (Express Server)

```
const express = require('express');
const app = express();

// set EJS
app.set('view engine', 'ejs');
// middleware to read form data
app.use(express.urlencoded({ extended: true }));
// render contact form
app.get('/contact', (req, res) => {
  res.render('contact');
});
// handle form submission
app.post('/submit', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;

  // render result page with data
  res.render('result', { name, email });
});

app.listen(3000, () => {console.log("Server running on port 3000");});
```

views/result.ejs

```
<h2>
  Thank you <%= name %>, we have received your email <%= email %>.
</h2>
```

Example -2

Write an Express.js application to configure the Multer middleware to upload a file. Perform the following tasks: 1) Create an EJS file that displays a heading "Upload Document" and contains a form with: -One file input field -One submit button 2) Render this EJS file on the /home route. 3) Configure Multer to store uploaded files in a folder named uploads. 4) After successful upload, display a message on the /upload route showing the original name of the uploaded file.

App.js

```
const express = require('express');
const multer = require('multer');
const path = require('path');

const app = express();

// set EJS
app.set('view engine', 'ejs');

// configure storage
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'uploads'); // folder name
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + path.extname(file.originalname));
  }
});

const upload = multer({ storage: storage });

// render home page
app.get('/home', (req, res) => {
  res.render('upload');
});
```

```
// handle file upload
app.post('/upload', upload.single('document'), (req, res) => {
  res.send(`File uploaded successfully: ${req.file.originalname}`);
});

// start server
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000/home");
});
```

views/Upload.ejs

```
<!DOCTYPE html>
<html>
<head>
  <title>Upload Document</title>
</head>
<body>

<h2>Upload Document</h2>

<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="document" required><br><br>
  <button type="submit">Upload</button>
</form>

</body>
</html>
```

Cookie

Example-1

You have been assigned to develop a user feedback form for a website using Express.js and cookies.

Implement the following requirements:

- Create a form with the following fields:
 - Name (input field)
 - Email (input field)
 - Message (textarea field)
 - Rating (radio buttons: Bad, Average, Good, Very Good, Excellent)
- When the user submits the form, store their feedback information (name, email, message, and rating) in a cookie named "feedback" that expires in 10 seconds.
- Display a confirmation message to the user after successfully submitting the form & Create a link to display the feedback details stored in the "feedback" cookie.
- When the user click to the link, retrieve the feedback information from the cookie and display it on the page also include a link on the feedback details page to Logout.
- When the user clicks the link, user redirected to home page.

In cookietask2.html file:

```
<html>
<head>
  <title>User Feedback Form</title>
</head>
<body>
  <h1>User Feedback Form</h1>
  <form action="/submit-feedback" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required><br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required><br><br>

    <label for="message">Message:</label>
    <textarea id="message" name="message" required></textarea><br><br>

    <label for="rating">Rating:</label>
    <input type="radio" id="rating1" name="rating" value="Bad" required>
    <label for="rating1">Bad</label>
    <input type="radio" id="rating2" name="rating" value="Average" required>
    <label for="rating2">Average</label>
    <input type="radio" id="rating3" name="rating" value="Good" required>
    <label for="rating3">Good</label>
    <input type="radio" id="rating4" name="rating" value="Very Good" required>
    <label for="rating4">Very Good</label>
    <input type="radio" id="rating5" name="rating" value="Excellent" required>
```

```

<label for="rating5">Excellent</label><br><br>

<input type="submit" value="Submit">
</form>
</body>
</html>

```

In task_cookie2.js file:

```

const express = require('express');
const cp = require('cookie-parser');
const app = express();
app.use(cp());
app.use(express.urlencoded({ extended: true }));

app.get('/', (req, res) => {
  res.sendFile(__dirname + '/cookietask2.html');
});

app.post('/submit-feedback', (req, res) => {
  const { name, email, message, rating } = req.body;

  // Create the feedback object
  const feedback = {
    name,
    email,
    message,
    rating
  };

  // Set the feedback cookie with a 10-second expiration
  res.cookie('feedback', feedback, { maxAge: 10000 });

  res.send('Thank you for your feedback! <br> <a href="/feedback-details"> Show Feedback </a>');
});

app.get('/feedback-details', (req, res) => {
  const feedback = req.cookies.feedback;

  if (feedback) {
    res.send(`
      <h1>Feedback Details</h1>
      <p><strong>Name:</strong> ${feedback.name}</p>
      <p><strong>Email:</strong> ${feedback.email}</p>
      <p><strong>Message:</strong> ${feedback.message}</p>
    `);
  }
});

```

```
    <p><strong>Rating:</strong> ${feedback.rating}</p>
    <a href="/" > logout </a>`);
  }
else {
  res.send('No feedback available.');
```

```
  }
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Example -2

Write an ExpressJS script to perform the following tasks:

Create one HTML file that contains:

Product Price (number input),Discount Percentage (number input),One dropdown (Festival Offer, Seasonal Offer, Clearance Sale),One submit button

Validation Rules:

Price and Discount must be greater than 0

Otherwise display message:"Enter valid price and discount"

User must select an offer from dropdown

Otherwise display:"Please select an offer type"

(Messages will be displayed on /discount page)

If valid: Store Price and Discount in cookies (expiry 60 seconds)

Display final price after discount on /discount page.

```
const express = require('express');
const cookieParser = require('cookie-parser');

const app = express();

app.use(cookieParser());

// serve HTML file
app.use(express.static(__dirname));
```

```
// handle request
app.get('/discount', (req, res) => {
  const price = parseFloat(req.query.price);
  const discount = parseFloat(req.query.discount);
  const offer = req.query.offer;

  // validation 1: price & discount > 0
  if (!price || !discount || price <= 0 || discount <= 0) {
    return res.send("<h2>Enter valid price and discount</h2>");
  }

  // validation 2: dropdown selected
  if (!offer) {
    return res.send("<h2>Please select an offer type</h2>");
  }

  // store cookies (60 seconds)
  res.cookie('price', price, { maxAge: 60000 });
  res.cookie('discount', discount, { maxAge: 60000 });

  // calculate final price
  const finalPrice = price - (price * discount / 100);

  res.send(`
    <h2>Offer Applied: ${offer}</h2>
    <p>Original Price: ${price}</p>
    <p>Discount: ${discount}%</p>
    <h3>Final Price: ${finalPrice}</h3>
  `);
});

// start server
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Discount Calculator</title>
</head>
<body>
```

```
<h2>Discount Form</h2>

<form action="/discount" method="GET">
  Product Price:
  <input type="number" name="price"><br><br>

  Discount (%):
  <input type="number" name="discount"><br><br>

  Offer Type:
  <select name="offer">
    <option value="">--Select Offer--</option>
    <option>Festival Offer</option>
    <option>Seasonal Offer</option>
    <option>Clearance Sale</option>
  </select><br><br>

  <input type="submit" value="Calculate">
</form>

</body>
</html>
```


Session

Example-1

Develop a web application using Node.js and Express.js to manage course registration details using sessions.

The application should display a form to collect Student Name, Course Name, and Number of Credits. After submission, store this data in a session and redirect to another page to validate the request.

If the number of credits is less than or equal to 6, display a success message with registered course details and a logout option. Otherwise, display an error message with a link to return to the form.

On logout, destroy the session and redirect to the home page.

```
const express = require('express');
const session = require('express-session');

const app = express();

app.use(express.urlencoded({ extended: true }));

app.use(session({
  secret: 'coursesecret',
  resave: false,
  saveUninitialized: true
}));

// serve form page
app.use(express.static(__dirname, { index: 'course.html' }));

// save session
app.post('/register', (req, res) => {
  const { name, course, credits } = req.body;

  req.session.name = name;
  req.session.course = course;
  req.session.credits = credits;

  res.redirect('/details');
});

// validate and display
```

```
app.get('/details', (req, res) => {
  const { name, course, credits } = req.session;

  if (credits <= 6) {
    res.send(`
      <h2>Course Registered Successfully</h2>
      <p>Student Name: ${name}</p>
      <p>Course: ${course}</p>
      <p>Credits: ${credits}</p>
      <a href="/logout">Logout</a>
    `);
  } else {
    res.send(`
      <h2>Cannot register for more than 6 credits</h2>
      <a href="/">Go Back</a>
    `);
  }
});

// destroy session
app.get('/logout', (req, res) => {
  req.session.destroy(() => {
    res.send(`
      <h2>Session destroyed</h2>
      <a href="/">Home</a>
    `);
  });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Course.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Course Registration</title>
</head>
<body>

<h2>Course Registration Form</h2>

<form action="/register" method="POST">
  Student Name: <input type="text" name="name" required><br><br>
```

```

Course Name: <input type="text" name="course" required><br><br>
Credits: <input type="number" name="credits" required><br><br>
<input type="submit" value="Register Course">
</form>

</body>
</html>

```

Example-2

write a script to meet following requirements (Authentication)

- 1)create index.html file page and open it on localhost.
- 2)after clicking submit button, it should jump on “savesession” page. Store username in session.
- 3)After saving session, redirect to “fetchsession” page and read value. Put a LOGOUT link button here.
- 4)Jump on delete session page after clicking LOGOUT button.
- 5)Destroy the session on this page and redirect to index.html page.

index.html

```

<html>
  <body> <center>
    <form method="get" action="/savesession">
      <div><input type = "text" name="uname" placeholder="Username" ></input></div>
      <div><input type = "password" name="pass" placeholder="Password"></input></div>
      <input class="lj_in" type="submit" name="submit" value="Submit">
    </form>
  </center> </body> </html>

```

Session.js

```

var expr = require("express");
var app = expr();
var es = require("express-session");
app.use(es({
  resave:false,
  saveUninitialized:false,
  secret:"ljul"
}));
app.use(expr.static("../public"))
app.get("/savesession",(req,res)=>{
  var uname=req.query.uname;
  var pass=req.query.pass;

  req.session.uname = uname;
  req.session.password = pass;
  res.redirect("fetchsession")

```

```

})
app.get("/fetchsession",(req,res)=>{
  if(req.session.uname == "admin" && req.session.password=="admin@123"){
    res.write("<h1 style='color:green;'>Welcome Admin</h1>")
    res.write("<a href='/deletesession'>Delete Session</a>")
  }else{
    res.write("<h1 style='color:red;'>Please enter valid username or password</h1><a
href='/>Login</a>")
  }
  res.send();
});
app.get("/deletesession",(req,res)=>{
  req.session.destroy();
  res.write("<h1>Session Destroyed</h1><a href='/>Login</a>")
  res.send();
});
app.listen(6177);

```

Example-3

Write a script to meet following requirements:

- Create session.html file page which contains form (username,password,login button). and open it on localhost.
- After clicking submit button, it should jump on “**save**” page. Store username and password in session.
- After saving session, redirect to “fetchdata” page and read value. Put a LOGOUT link button here. On this page check authentication of user. User name and password must be “admin” and “admin@123” respectively.
 - If this condition is true then display welcome admin and display logout link on this page(fetchdata).
- By clicking on logout link user should jump to “destroy” page and destroy the session there and display the message “Session destroyed”. And give the link of “login” under that message. By clicking that link user will be redirected to the home page.
 - Else display “Please enter valid username and password” and login link on this page(fetchdata).

session.html

```

<html> <body>
  <center>
    <form method="get" action="/savesession">

      <div><input type = "text" name="uname" placeholder="Username"
></input></div>
      <div><input type = "password" name="pass"
placeholder="Password"></input></div>
      <input class="lj_in" type="submit" name="submit" value="Login">
    </form>
  </center>

```

```
</body> </html>
```

Session.js

```
var expr = require("express");
var app = expr();
var es = require("express-session");
app.use(es({
  resave:false,
  saveUninitialized:false,
  secret:"lju1"
}));
app.get('/',(req,res)=>{
  res.sendFile(__dirname + "/session.html");
});
app.get("/save",(req,res)=>{
  var uname=req.query.uname;
  var pass=req.query.pass;

  req.session.uname = uname;
  req.session.password = pass;
  res.redirect("fetchsession")
})
app.get("/fetchdata",(req,res)=>{
  if(req.session.uname == "admin" && req.session.password=="admin@123"){
    res.write("<h1 style='color:green;'>Welcome Admin</h1>")
    res.write("<a href=/destory>Logout</a>")
  }else{
    res.write("<h1 style='color:red;'>Please enter valid username or password</h1><a href=/>Login</a>")
  }
  res.send();
});
app.get("/destroy",(req,res)=>{
  req.session.destroy(err => {
    if (err) {
      res.send(err)
    }
    else {
      res.clearCookie('connect.sid')
      res.redirect('/')
    }
  }) });
app.listen(6177);
```

Middleware

Example-1

Develop a web application using Express.js that processes user input through middleware functions. The application should accept two numbers (num1, num2) and an operation (add, square, or cube) via a POST request to the /calc route.

The first middleware should validate that both numbers are greater than 0 and calculate their sum. The second middleware should perform the selected operation on the sum (returning either the sum, its square, or its cube). If validation fails, an error message should be displayed.

Finally, the server should send the computed result back to the user in a proper response format.

```
var express = require('express');
var app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.static(__dirname, { index: "my.html" }));
// Middleware 1: Calculate sum
function calcMiddleware(req, res, next) {
  let num1 = parseInt(req.body.num1);
  let num2 = parseInt(req.body.num2);
  req.op = req.body.operation;
  if (num1 <= 0 || num2 <= 0) {
    res.send("Numbers must be greater than 0");
  }
  req.sum = num1 + num2;
  next();
}
// Middleware 2: Perform operation
function moveBy(req, res, next) {
  let result;
  if (req.op === "add") {
    result = req.sum;
  } else if (req.op === "square") {
    result = req.sum * req.sum;
  } else if (req.op === "cube") {
    result = req.sum * req.sum * req.sum;
  }
  req.result = result;
  next();
}
// Route using BOTH middlewares
app.post('/calc', calcMiddleware, moveBy, function(req, res) {
  res.send("Result is: " + req.result);
});
app.listen(3000);
```

my.html

```
<form action="/calc" method="POST">
  <input type="number" name="num1" placeholder="Enter number 1" required min="1">
  <input type="number" name="num2" placeholder="Enter number 2" required min="1">

  <select name="operation">
    <option value="add">Addition</option>
    <option value="square">Square of Sum</option>
    <option value="cube">Cube of Sum</option>
  </select>

  <button type="submit">Calculate</button>
</form>
```

Example-2

Write express JS script to load in html file having username, password and submit button. On clicking of submit button, it should jump on check page using post method. If username is “admin” then jump on next middleware to print “welcome admin”. For any other username it should stay on same middleware to print warning message in red color.

two.html (file in public folder)

```
<form action="/check" method="post">
<fieldset>
<legend>Sign-up</legend>
  Username: <input type="text" name="uname"/>
</br></br>
  password: <input type="text" name="pwd"/>
<br></br>
<input type="submit"/>
</fieldset>
</form>
```

task.js (file in src folder)

```
const expr=require("express");
const app=expr();

app.use(expr.urlencoded({extended:false}))
app.use(expr.static('../public',{index:"two.html"}));
```

```

app.post("/check",(req,res,next)=>{
  const u1=req.body.uname
  if(u1=="admin")  {
    next();
  }
  else  {
    res.write(`<h1 style="color:red"> You are not admin </h1>`);
    res.send();
  } } );
app.post("/check",(req,res)=>{
  res.write("<h1>welcome admin</h1>");
  res.send()  });
app.listen(7002)

```

Example-3

Write express js script to perform following tasks. (Middleware)

Create one html file which contains one text field for name, email field and checkbox for subscription. Html file will be loaded on home page. Email and name fields are required fields.

On login page welcome user and email id data should be printed.

- If user checked the subscription then “Thank you for the subscription” message will be printed and “logout” link will be displayed under the message. If user clicks logout link then he/she will be redirected to the home page.
- If user has not opted for the subscription then “You can subscribe to get daily updates” message will be printed and “subscribe” link will be displayed under the message.
- If user clicks subscribe link then he/she will be redirected to the subscription page. In this page “Thank you for the subscription” message will be printed and “logout” link will be displayed under the message. If user clicks logout link then he/she will be redirected to the home page.

Use concept of the middleware and you can use any of http methods(get/post).

/src/app1.js

```

var expr = require("express");
var app = expr();
app.use(expr.static('../public',{index:'trial1.html'}));
app.get("/login",(req,res,next)=>{
  console.log(req.query);
  res.set("content-type","text/html");
  res.write("<center><h1>Welcome " + req.query.name + "</h1>");
  res.write("<center><h2>Your email id is " + req.query.email + "</h2>");

```



```

    next();
  })
  app.get("/login",(req,res,next)=>{
    if(req.query.newsletter && req.query.newsletter == "on"){
      res.write("<h3>Thank you for your subscription</h3><a href='/'>Logout</a>");
    }else{
      res.write("<h3>You can subscribe to get daily updates</h3><a
href='/subscribe'>Subscribe</a></center>");
    }
    next();
  });
  app.get("/subscribe",(req,res)=>{
    res.set("content-type","text/html");
    res.write("<h3>Thank you for your subscription</h3></center><a href='/'>Logout</a>");
    res.send();
  });
  app.listen(5001);

```

/public/trial1.html

```

<form action="/login" method="get">
  <label for="name">Name:</label>
  <input type="text" name="name" required><br><br>

  <label for="email">Email:</label>
  <input type="email" name="email" required><br><br>

  <label for="newsletter">Subscribe?</label>
  <input type="checkbox" name="newsletter" id="newsletter"><br><br>

  <input type="submit" value="Submit">
</form>

```

← → ↻ ⓘ localhost:5001

Welcome lju

Your email id is lju@gmail.com

Thank you for your subscription

☐ **Subscribe?**

[Logout](#)

Welcome lju

Your email id is lju@gmail.com

You can subscribe to get daily updates

[Subscribe](#)

Example-4

Write an ExpressJS to take a UserName, Password, Textarea for “message” & submit button using get method.

- 1) After clicking submit button the content of submitted details should be represented on “/login” page along with one “show vowel” link.
- 2) By clicking “show vowel” link count of vowel used in submitted “message” will display on “/message” page. (Use next() to route page)

```
const express = require("express");
const app = express();

// Home page
app.get("/", (req, res) => {
  res.send(`
    <h2>Form</h2>
    <form action="/login" method="get">
      Username: <input type="text" name="username"><br><br>
      Password: <input type="password" name="password"><br><br>
      Message:<br>
      <textarea name="message"></textarea><br><br>
      <button type="submit">Submit</button>
    </form>
  `);
});

// middleware
function showData(req, res, next) {
  res.type("text/html");
  let u = req.query.username;
  let p = req.query.password;
  let m = req.query.message;
  res.write("Username: " + u + "<br>");
  res.write("Password: " + p + "<br>");
  res.write("Message: " + m + "<br><br>");
  res.write(`<a href="/message?message=${m}">Show Vowel</a><br><br>`);
  next();
}

// route
app.get("/login", showData, (req, res) => {
  res.send("Submitted");
});

// message page
app.get("/message", (req, res) => {
  let msg = req.query.message;
  let count = 0;

  for (let ch of msg.toLowerCase()) {
```

```
        if ("aeiou".includes(ch)) { count++;}  
      }  
      res.send("Vowel Count: " + count);  
    });  
app.listen(3000);
```

Get/ Post methods

Example-1

Write an ExpressJS application that creates a login form containing fields for username and password along with a submit button.

The form data should be sent to the server using the POST method. Implement an authentication middleware that checks whether the entered username is "admin" and the password is "1234".

If the credentials are correct, the application should display the message "Login Successful". Otherwise, it should display "Invalid Credentials".

```
const express = require("express");
const app = express();
// to read POST data
app.use(express.urlencoded({ extended: true }));
// form page
app.get("/", (req, res) => {
  res.send(`
    <h2>Login Form</h2>
    <form action="/login" method="post">
      Username: <input type="text" name="username"><br><br>
      Password: <input type="password" name="password"><br><br>
      <button type="submit">Login</button>
    </form>
  `);
});
// middleware for authentication
function auth(req, res, next) {
  let u = req.body.username;
  let p = req.body.password;
  if (u === "admin" && p === "1234") {
    next(); // correct
  } else {
    res.send("Invalid Credentials");
  }
}
// route
app.post("/login", auth, (req, res) => {
  res.send("Login Successful");
});
// server
app.listen(3000);
```

Example-2

Write an ExpressJS application that creates an HTML form containing two text fields named name and age, along with a submit button. When the user submits the form, the data should be sent using the GET method. After submission, the application should redirect to a new page and display the message:

"Hello <name>, your age is <age>", where the values are taken from the form input.

```
const express = require("express");
const app = express();
// form page
app.get("/", (req, res) => {
  res.send(
    <h2>User Form</h2>
    <form action="/data" method="get">
      Name: <input type="text" name="name"><br><br>
      Age: <input type="text" name="age"><br><br>
      <button type="submit">Submit</button>
    </form>
  );
});
// redirect route
app.get("/data", (req, res) => {
  let name = req.query.name;
  let age = req.query.age;
  // redirect to new page
  res.redirect(`/result?name=${name}&age=${age}`);
});
// result page
app.get("/result", (req, res) => {
  let name = req.query.name;
  let age = req.query.age;

  res.send(`Hello ${name}, your age is ${age}`);
});
// server
app.listen(3000);
```

Example-3

Write express js script to perform tasks as asked below. **(Get Method)**

1. Create one HTML file which contains two number type input fields, one dropdown which contains options like (select, addition, subtraction, multiplication, division) and one submit button.
2. The input fields must contain the value greater than 0 else it will give a message "Please enter the valid message". Also, user must select any of the formula from the dropdown else give a message "You have not selected any formula". (Message will be displayed on "/calc" page.)
3. If one formula is selected and numbers are entered then respective calculations will be performed on the page "/calc".
4. Use get method to request data.

/src/calc.js

```
var expr = require("express");
var app = expr();
app.use(expr.static("../css"));
app.use(expr.static("../html",{index:'1.html'}));

app.get("/calc",(req,res,next)=>{
  res.set("content-type","text/html");
  var n1 = parseInt(req.query.n1);
  var n2 = parseInt(req.query.n2);
  if ((n1>0) && (n2>0)) {
    if(req.query.formula == "addition"){
      a = n1+n2;
      res.write("<h1>Addition is : " + a + "</h1>");
    }
    else if(req.query.formula == "subtraction"){
      s = n1-n2;
      res.write("<h1>Subtraction is : " + s + "</h1>");
    }
    else if(req.query.formula == "multi"){
      m = n1*n2;
      res.write("<h1>Multiplication is : " + m + "</h1>");
    }
    else if(req.query.formula == "div"){
      d = n1/n2;
      res.write("<h1>Division is : " + d + "</h1>");
    }
    else{
      res.write("<h1>You have not selected any formula.</h1>");
    }
  }else{
    res.write("<h1>Please enter the valid number/s.</h1>");
  }
  res.send()
})
```

```
})  
app.listen(6012);  
/css/t1.css
```

```
.lj_form{  
  background-color: rgb(158, 158, 219);  
}
```

/html/1.html

```
<html><head>  
  <link rel="stylesheet" href="/t1.css">  
</head>  
<body>  
  <form action="/calc" method="get" class="lj_form">  
    <fieldset>  
      <legend>Calculator:</legend>  
      <label for="n1">Enter Number1:</label>  
      <input type="number" name="n1"><br><br>  
  
      <label for="n2">Enter Number2:</label>  
      <input type="number" name="n2"><br><br>  
  
      <label for="formula">Formula: </label>  
      <select name="formula" id="formula">  
        <option value="">Select formula</option>  
        <option value="addition">Addition</option>  
        <option value="subtraction">Subtraction</option>  
        <option value="multi">Multiplication</option>  
        <option value="div">Division</option>  
      </select><br><br>  
  
      <input type="submit" value="Submit">  
    </fieldset>  
  </form>  
</body></html>
```

Output:



Addition is : 8

Example-1

Example-4

Write express script to get form data using post method and display First name in Red color, Password in Green Color and Gender in Black color.

trial.html (file in public folder)

```
<form method="post" action="/process-post">
<fieldset>
  <legend style="color:blue;text-align:center" >Log In </legend>
  </br>
  <label> User Name</label>
    <input type="text" name="firstname"/>
  </br>
  <label> Password</label>
    <input type="password" name="password"/>
  </br>
  <label> Gender</label>
    <input type="radio" name="r1" value="Male">Male</input>
    <input type="radio" name="r1" value="Female">Female</input>
  </br>
  <label> Comment</label>
    <textarea rows="5" cols="15" name="comment"> </textarea>
  </br>
    <input type="Submit">
    <input type="Reset">
</fieldset>
</form>
```

trial.js (file in src folder)

```
const expr=require("express")
const app=expr();

app.use(expr.urlencoded({extended:false}))

app.use(expr.static('../public',{index:"trial.html"}))
app.post("/process-post",(req,res)=> {
  res.write(`<h1 style="color:red">Firstname=${req.body.firstname}</h1>
    <h1 style="color:green">Password=${req.body.password}</h1>
    <h1>Gender=${req.body.r1}</h1>`)
  res.end()
})
app.listen(5000)
```



```
console.log("Server is running")
```

Output:

After filling form on localhost:5000 with name:"abc", password: "abcd", gender: Male

Output on: <http://localhost:5000/process-post>

Firstname=abc

Password=abcd

Gender=Male

Example-5

Write express js script to perform the tasks as asked below. (Post method)

- 1) Create one HTML file named ljform.html and add one form which contains username, password and submit button. Data should be submitted by HTTP post method.
- 2) Submit button is of black color with white text. (External CSS)
- 3) On home page form should be displayed and while submitting the form, on next page named "/login" if username is admin then it will display "Welcome admin" else display "Please login with Admin name".

In 3.js file in src folder

```
var expr = require("express");
var app = expr();
app.use(expr.urlencoded());
app.use(expr.static('./public',{index: 'ljform.html'}));
app.post("/login",(req,res)=>{
  if( req.body.uname == 'admin' ){
    res.write(<h1 style="color:green">Hey ${req.body.uname},Welcome!</h1><br>`)
  }
  else{
    res.write(<h1 style="color:red">Please login with Admin name</h1>`);
  }
  res.send();
})
app.listen(5222);
```

In 1.css file in public folder

```
.click{  
  background-color: black;  
  color: white  
}
```

In ljform.html file in public folder

```
<html>  
  <head>  
    <link rel="stylesheet" href="1.css">  
  </head>  
  <body>  
    <form method="post" action="/login">  
      <label>User name:</label>  
      <input type = "text" name="uname" ></input>  
      <label>Password:</label>  
      <input type = "text" name="password" ></input>  
      <button name="Submit" value="Submit" class="click">Submit</button>  
    </form>  
  </body>  
</html>
```

REST API

Example-1

Write an expressJS code in which RESTapi is created for json object named Places I love which contains name,country,state,city and rating out of 10(no decimal points) is given.upon passing ratings on the browser it should display the places having that rating.i.e. on localhost:30001/a/10 should display all the places having 10 ratings.

Solution:

api.js

```
const expr = require("express");
const router = expr.Router();
const PlacesIlove=[
  {name:'manali',country:'india',state:'jammu',city:'manali',rating:8},
  {name:'kullu',country:'india',state:'jammu',city:'manali',rating:9},
  {name:'kashmir',country:'india',state:'jammu',city:'manali',rating:10}]
module.exports = router;
router.get("/:rating([0-9]){1,2})",(req,res)=>{
  var current_data = data.filter((i1)=>{
    if(i1.rating == req.params.rating){
      return true;
    }
  })
  if(current_data.length > 0){
    res.send(current_data);
  } else {
    res.send("Not Found")
  }
})
```

main.js

```
const expr = require("express");
const app = expr();
const api = require("./api")
app.use("/a",api);
app.listen(30001);
```